

ECE264-003 Fall 2019 Exam 3
8-9 PM, Thursday, Nov 21st, 2019

1. Merge Two Sorted Linked Lists

The following is the definition of node in a singly linked list.

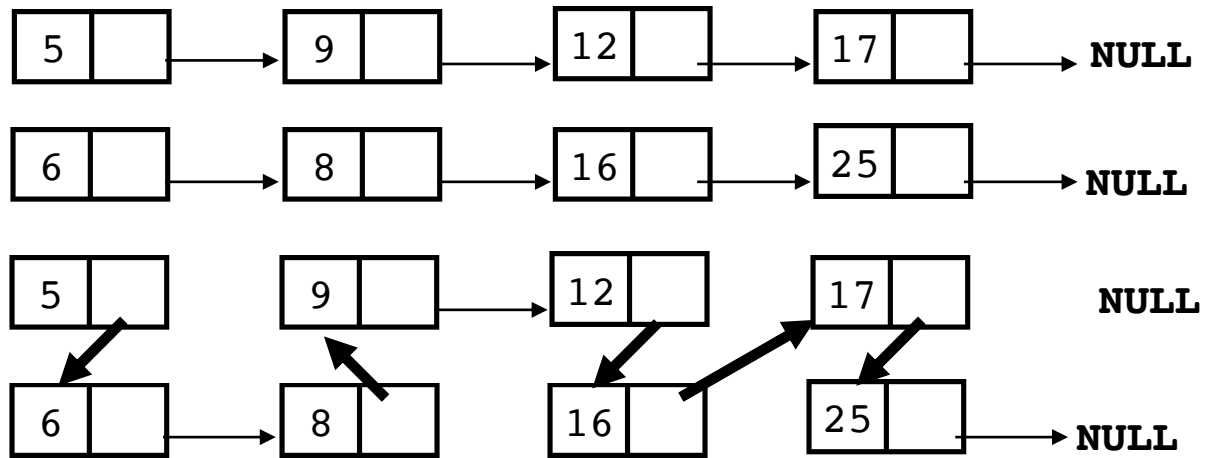
```
1.  typedef struct node
2.  {
3.      struct node * next;
4.      int value;
5.  } ListNode;
```

Consider the following recursive function to merge two sorted linked lists to one single sorted linked list:

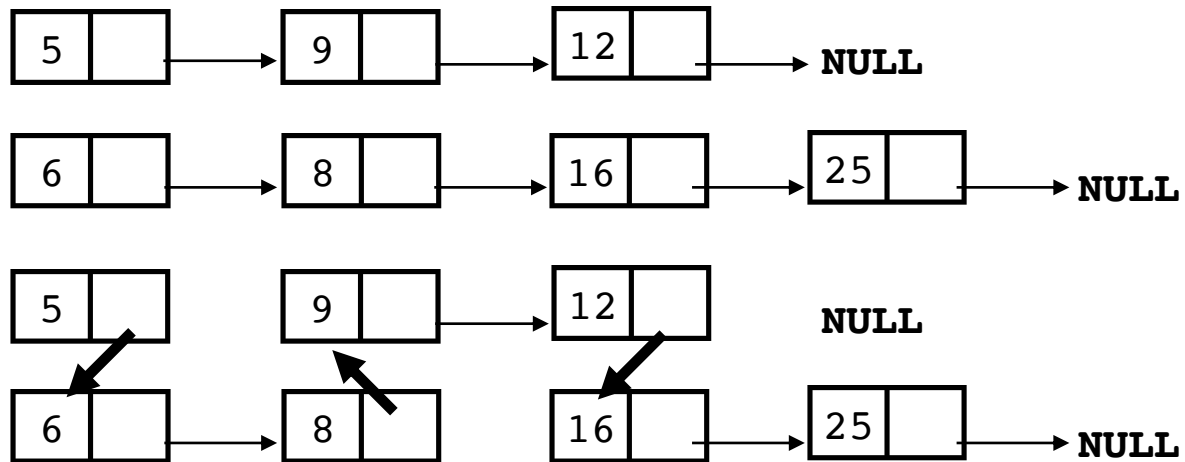
```
1.  ListNode * MergeSortedList(ListNode * p, ListNode * q)
2.  {
3.      ListNode * head = NULL;
4.      if (p == NULL)
5.      {
6.          return q;
7.      }
8.      if (q == NULL)
9.      {
10.         return p;
11.     }
12.     if (p->value <= q->value)
13.     {
14.         head = p;
15.         head->next = MergeSortedList(p->next, q);
16.     }
17.     else
18.     {
19.         head = q;
20.         head->next = MergeSortedList(p, q->next);
21.     }
22.     return head;
23. }
```

The function `MergeSortedList` merges the two sorted linked lists by modifying some links.

For example, merging the following two sorted linked lists needs to modify five links.



Merging the following two sorted linked list needs to modify three links.



Suppose the two sorted linked lists have distinct values (every value in the linked lists is unique). Denote the number of nodes in the first sorted linked list as m , and the number of nodes in the second sorted linked list as n .

1.A: If $m = 3$, $n = 3$, what is the **minimum** number of modified links?

1.B: If $m = 3$, $n = 3$, what is the **maximum** number of modified links?

1.C: If $m = 6$, $n = 6$, what is the **maximum** number of modified links?

1.D: If $m = 5$, $n = 6$, what is the **minimum** number of modified links?

1.E: If $m = 5$, $n = 6$, what is the **maximum** number of modified links?

Answer:

1.A: 1

1.B: 5

1.C: 11

1.D: 1

1.E: 10

2. Arithmetic Expression

Question 2.A - 2.C: Evaluate the result of the following prefix or postfix expressions. Please answer **Invalid** if you find the expression to be invalid.

Hint: the expression is considered invalid if:

- 1) an operator is seen but there are less than two operands left in the stack
- 2) all operators have been seen but there are more than one operand left in the stack

2.A: 5 3 6 + * 9 - 8 2 + *

2.B: 18 4 + 7 3 - 5 * - 9 2 1 + +

2.C: - * 4 7 + * 3 6 - 8 5

Question 2.D - 2.E: Consider the following infix expression.

$(5 * (3 + 6) - 2) - (15 + 3)$

2.D: Convert the above infix expression to **postfix** expression. From the left to right, what is the **second operator** in the postfix expression?

2.E: Convert the above infix expression to **prefix** expression. From the left to right, what is the **first operator** in the prefix expression?

Answer:

2.A: 360

2.B: Invalid

2.C: 7

2.D: *

2.E: -

3. Inserting a Node to a Binary Search Tree

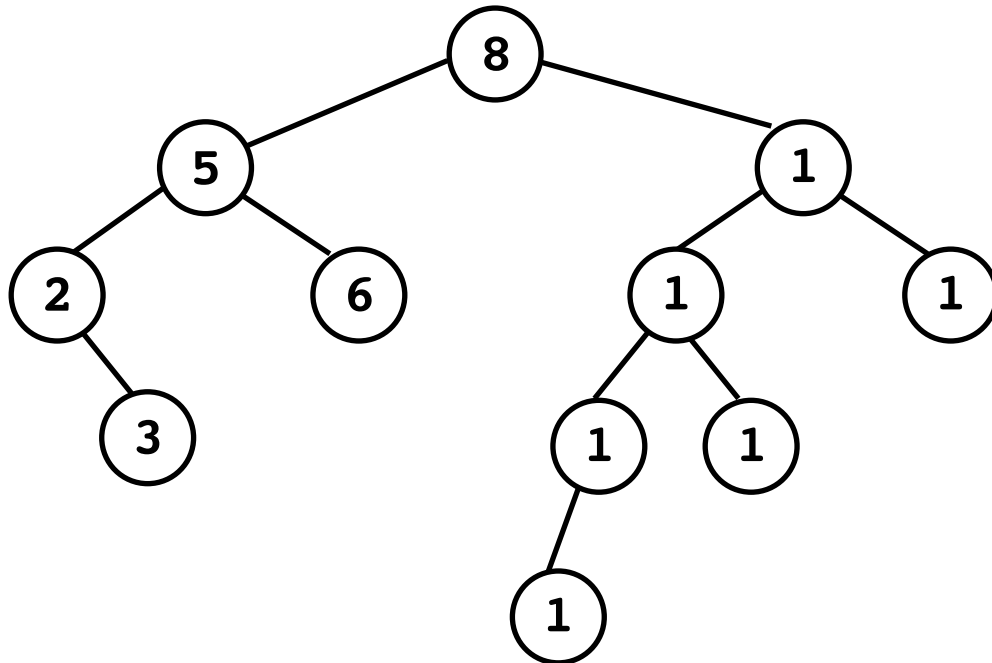
The following is the definition of node in a binary tree.

```
1.  typedef struct node
2.  {
3.      struct node * left;
4.      struct node * right;
5.      int value;
6.  } TreeNode;
```

The following is an incorrect function to insert node to a binary search tree. There is an error in the `TreeInsert` function. **Please don't fix the error.**

```
1.  static TreeNode * TreeNodeConstruct(int val)
2.  {
3.      TreeNode * tn;
4.      tn = malloc(sizeof(TreeNode));
5.      tn->left = NULL;
6.      tn->right = NULL;
7.      tn->value = val;
8.      return tn;
9.  }
10. TreeNode * TreeInsert(TreeNode * tn, int val)
11. {
12.     // if empty, create a node
13.     if (tn == NULL)
14.     {
15.         return TreeNodeConstruct(val);
16.     }
17.     // if not empty
18.     if (val == (tn->value))
19.     {
20.         // ERROR here: should return tn, but didn't return
21.         // please don't fix the code
22.     }
23.     if (val > (tn->value))
24.     {
25.         tn->right = TreeInsert(tn->right, val);
26.     }
27.     else
28.     {
29.         tn->left = TreeInsert(tn->left, val);
30.     }
31.     return tn;
32. }
```

We use the above `TreeInsert` function to insert node 17, 8, 15, 6, 7 (in this order) to the following binary search tree. After inserting the nodes, please answer the questions 3.A - 3.E.



- 3.A:** What is the **first** number in the **pre-order** traversal?
- 3.B:** What is the **eighth** number in the **pre-order** traversal?
- 3.C:** What is the **last** number in the **post-order** traversal?
- 3.D:** What is the **eighth** number in the **post-order** traversal?
- 3.E:** What is the **seventh** number in the **in-order** traversal?

Answer:

3.A: 8

3.B: 7

3.C: 8

3.D: 10

3.E: 8

pre-order traversal: 8 5 2 3 6 6 8 7 15 12 11 10 14 15 17 17

post-order traversal: 3 2 6 7 8 6 5 10 11 15 14 12 17 17 15 8

in-order traversal: 2 3 5 6 6 7 8 8 10 11 12 14 15 15 17 17

4. Shape of Binary Search Tree

The shape of binary search tree depends on the order of the input. For example, there are $3! = 6$ different orders of 1, 2, 3, which create 5 different shapes:

- If the input order is 1, 2, 3, the binary search tree is in Fig. 4 (a).
- If the input order is 1, 3, 2, the binary search tree is in Fig. 4 (b).
- If the input order is 2, 1, 3, the binary search tree is in Fig. 4 (c).
- If the input order is 2, 3, 1, the binary search tree is in Fig. 4 (c).
- If the input order is 3, 1, 2, the binary search tree is in Fig. 4 (d).
- If the input order is 3, 2, 1, the binary search tree is in Fig. 4 (e).

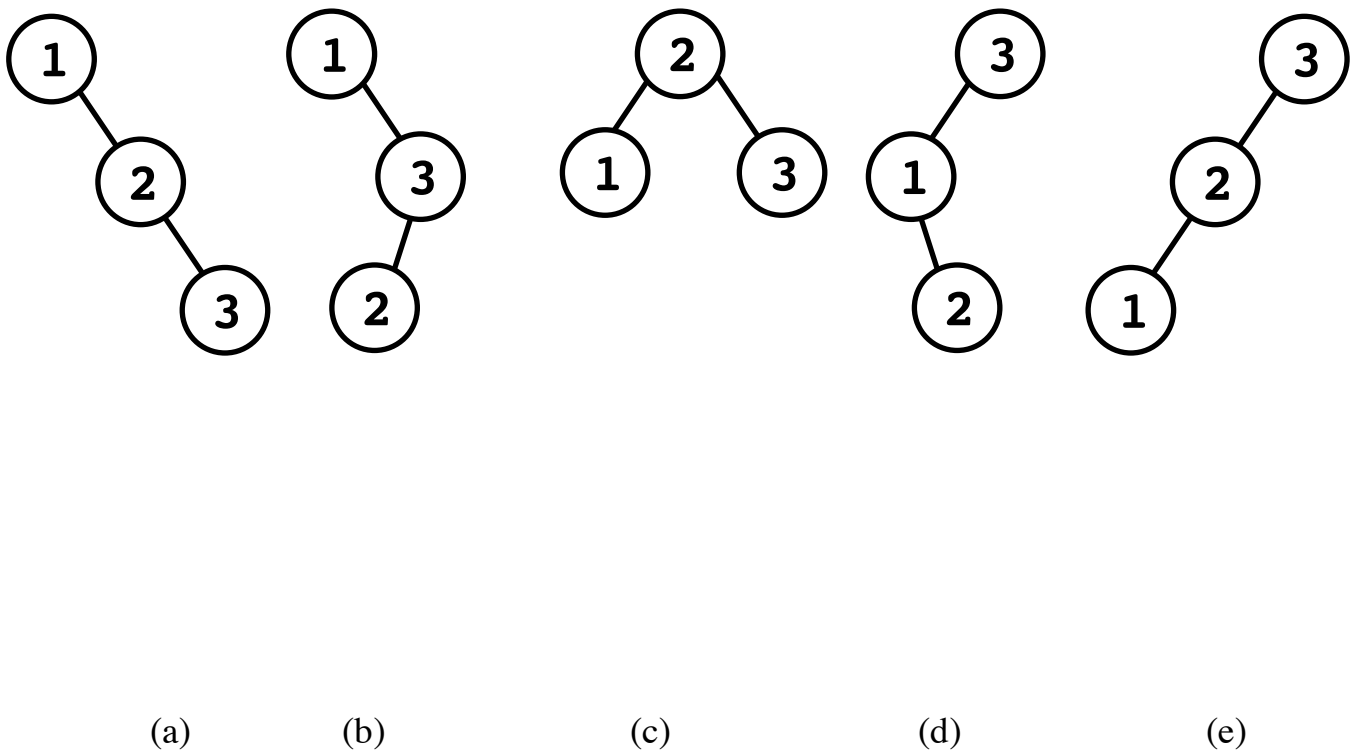


Figure 4: Binary search trees constructed from different input order of 1, 2, 3.

4.A: There are $4! = 24$ different orders of 1, 2, 3, 4. How many different binary search trees will these different orders create?

4.B: There are $5! = 120$ different orders of 1, 2, 3, 4, 5. How many different binary search trees will these different orders create?

4.C: There are $7! = 5040$ different orders of 1, 2, 3, 4, 5, 6, 7. How many different binary search trees will these different orders create, if the input order starts with 2 (*i.e.*, root node of the binary search tree is 2)?

4.D: There are $8! = 40320$ different orders of 1, 2, 3, 4, 5, 6, 7, 8. How many different binary search trees will these different orders create, if the input order starts with 4 (*i.e.*, root node of the binary search tree is 4)?

4.E: If a **binary search tree** is created with a certain order of 1, 2, 3, 4, 5, 6, 7, 8, and the post-order traversal of this binary search tree is 2, 1, 4, 5, 8, 7, 6, 3. What is the **fourth** number in the **pre-order** traversal of this binary search tree?

Answer:

4.A: 14

4.B: 42

4.C: 42

4.D: 70

4.E: 6